Experiment No. 2 (Software) **Date: 28-08-2022**

DESIGN OF ADDER AND SUBTRACTOR USING LOGIC GATES

Register Number: 21BEC1851

Name: Rahul Karthik S

Aim:

To design and construct half adder, full adder, half subtractor, full subtractor circuits using logic gates in Verilog and verify its truth table.

Software Required:

ModelSim

Theory:

Half Adder:

A half adder has two inputs for the two bits to be added and two outputs one from the sum 'S' and other from the carry 'c' into the higher adder position. Above circuit is called as a carry signal from the addition of the less significant bits sum from the X-OR Gate the carry out from the AND gate.

Full Adder:

A full adder is a combinational circuit that forms the arithmetic sum of input; it consists of three inputs and two outputs. A full adder is useful to add three bits at a time but a half adder cannot do so. In full adder sum output will be taken from X-OR Gate, carry output will be taken from OR Gate.

Half Subtractor:

The half subtractor is constructed using X-OR and AND Gate. The half subtractor has two input and two outputs. The outputs are difference and borrow. The difference can be applied using X-OR Gate, borrow output can be implemented using an AND Gate and an inverter.

Full Subtractor:

The full subtractor is a combination of X-OR, AND, OR, NOT Gates. In a full subtractor the logic circuit should have three inputs and two outputs. The two half subtractor put together gives a full subtractor. The first half subtractor will be C and AB. The output will be difference output of full subtractor. The expression AB assembles the borrow output of the half subtractor and the second term is the inverted difference output of first X-OR.

Procedure:

1. Click Jumpstart, create new project and type the project name and click ok.

2. Click Create New File, type the name and click Verilog from the drop-down and click ok.
3. After typing the code, click compile, compile selected from menu.
4. Click Simulate and click start simulation.
5. Select the variables and Click add wave.
6. For input, select the input using Force or Clock and run it.

Design and Logic Diagram:

<u>Half Adder:</u>

Truth Table:

A	**B**	**CARRY**	**SUM**
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

K- Map for SUM:

A \ B	00	01
00		1
01	1	

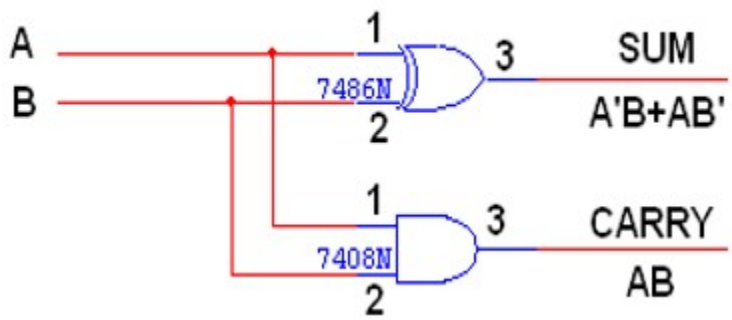
SUM = A'B + AB' = A $\oplus$ B

K- Map for CARRY:

A \ B	00	01
00		
01		1

CARRY = AB

Truth Table:



<u>Full Adder:</u>

Truth Table:

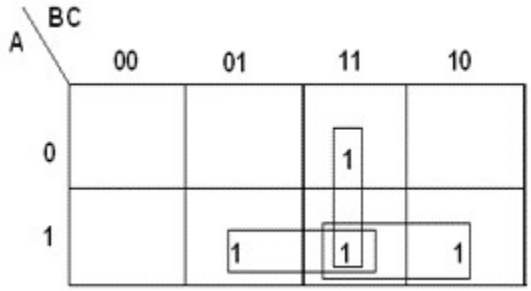
A	**B**	**C**	**CARRY**	**SUM**
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

K- Map for SUM:

A \ BC	00	01	11	10
0		1		1
1	1		1	

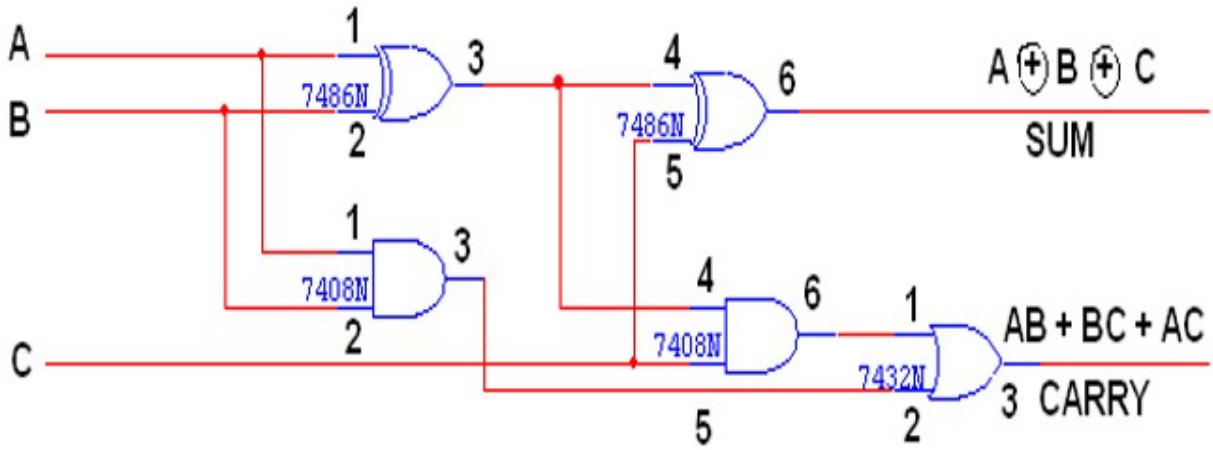
SUM = A'B'C + A'BC' + ABC' + ABC = A ⊕ B ⊕ C

K- Map for CARRY:



CARRY = AB + BC + AC

Truth Table:

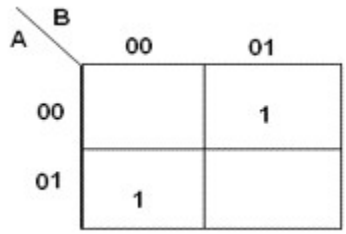


<u>Half Subtractor:</u>

Truth Table:

A	B	BORROW	DIFFERENCE
0	0	0	0
0	1	1	1
1	0	0	1
1	1	0	0

K- Map for DIFFERENCE:



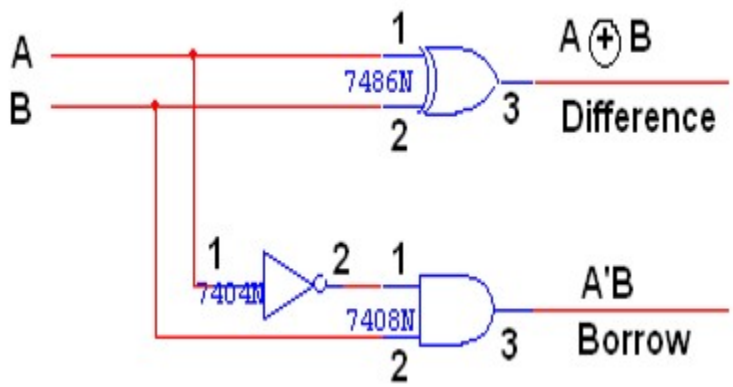
DIFFERENCE = A’B + AB’ = A $\oplus$ B

K- Map for BORROW:

A \ B	00	01
00		1
01		

BORROW = A’B

Truth Table:



<u>Full Subtractor:</u>

Truth Table:

A	B	C	BORROW	DIFFERENCE
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	1	0
1	0	0	0	1
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

K- Map for DIFFERENCE:

A \ BC	00	01	11	10
0		1		1
1	1		1	

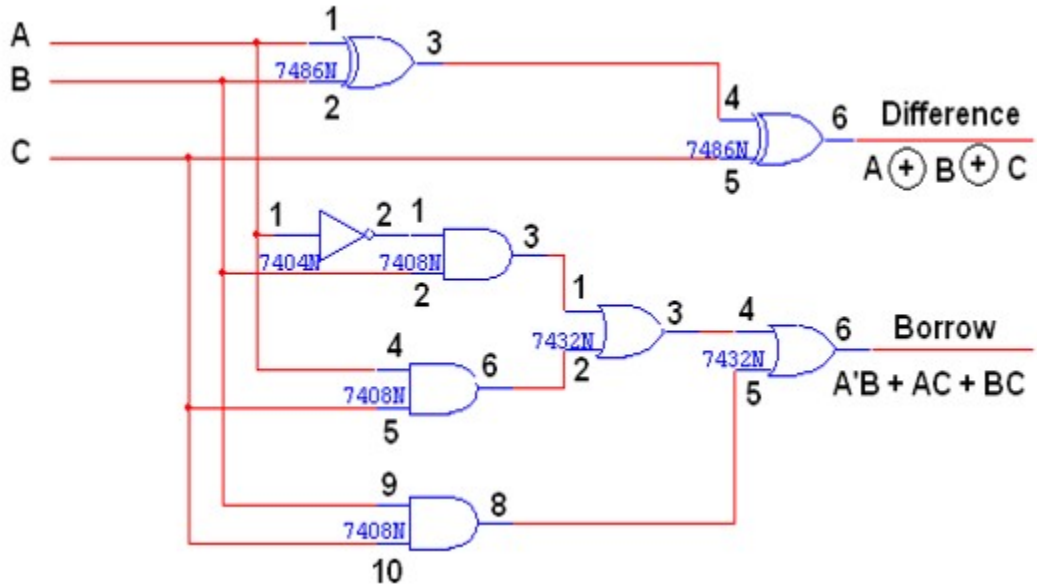
DIFFERENCE = A’B’C + A’BC’ + AB’C’ + ABC

K- Map for BORROW:

A \ BC	00	01	11	10
0		1	1	1
1			1	

BORROW = A’B + BC + A’C

Truth Table:

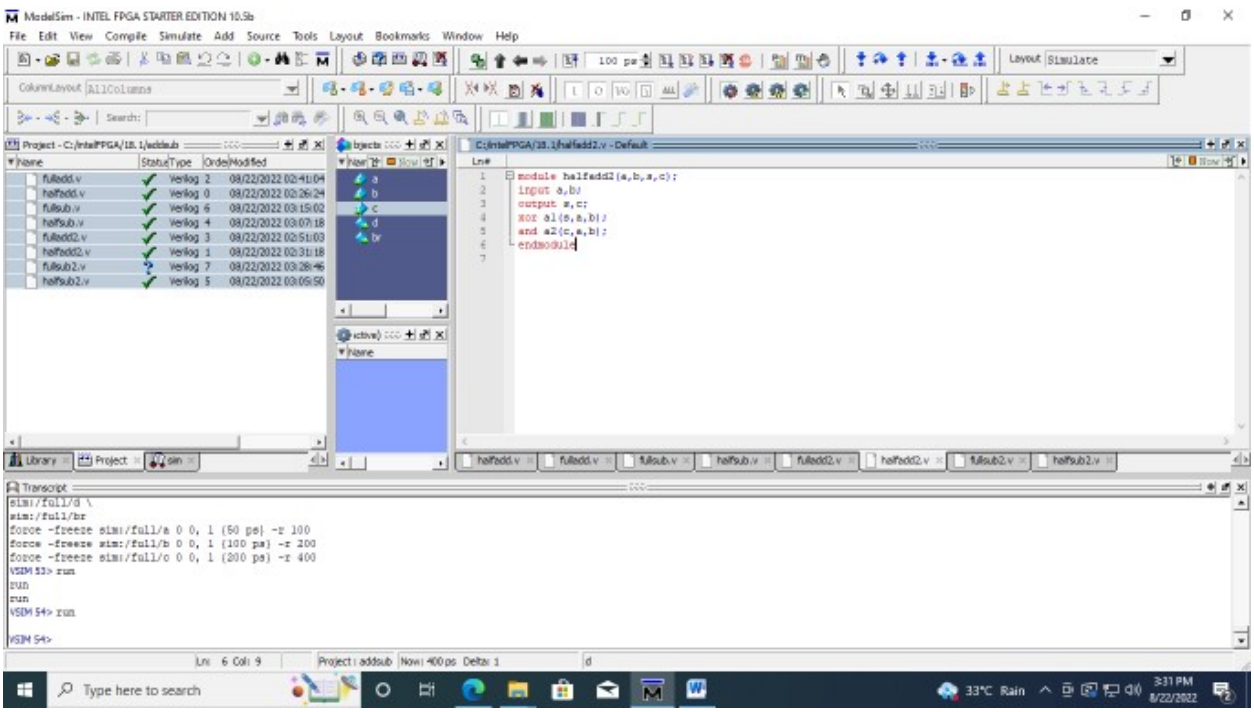


Verilog Code:

<u>Gate Modelling:</u>

Half Adder:

```
module halfadd2(a,b,s,c);

input a,b;

output s,c;

xor a1(s,a,b);

and a2(c,a,b);

endmodule
```

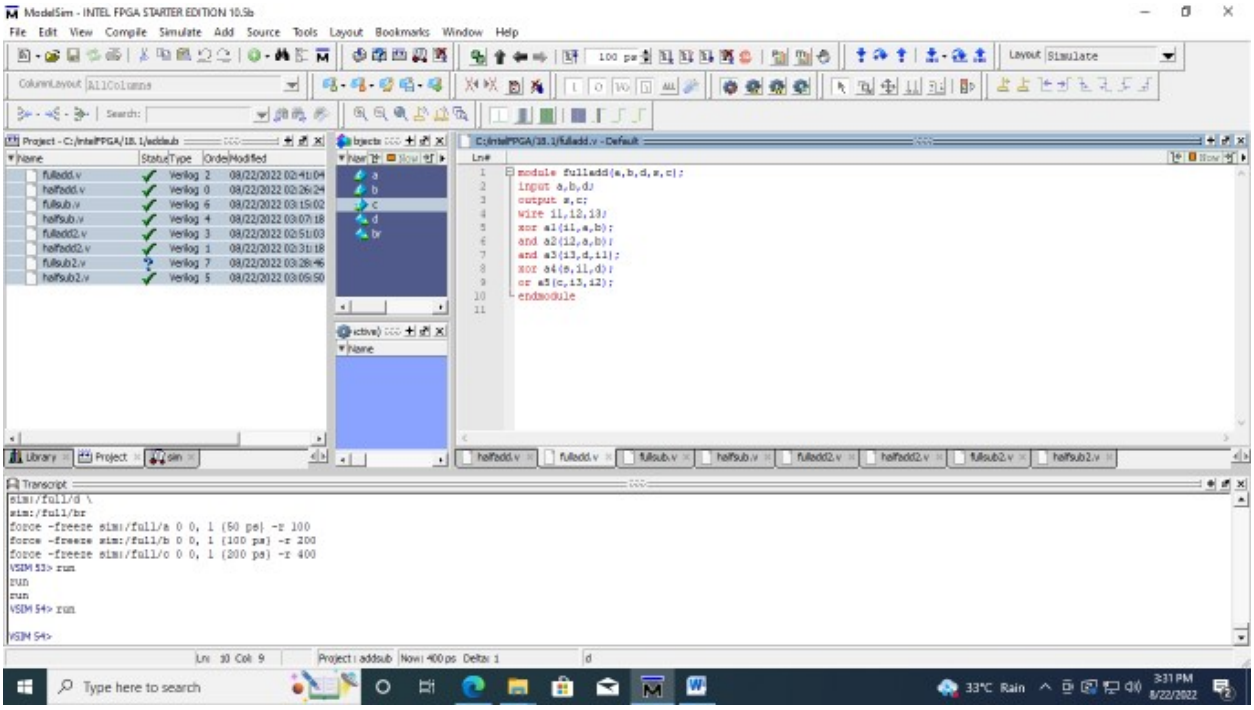


Full Adder:

```
module fulladd(a,b,d,s,c);

input a,b,d;

output s,c;

wire i1,i2,i3;

xor a1(i1,a,b);

and a2(i2,a,b);

and a3(i3,d,i1);

xor a4(s,i1,d);
```

or a5(c,i3,i2);

endmodule



Half Subtractor:

module halfsub(a,b,d,br);

input a,b;

wire i1;

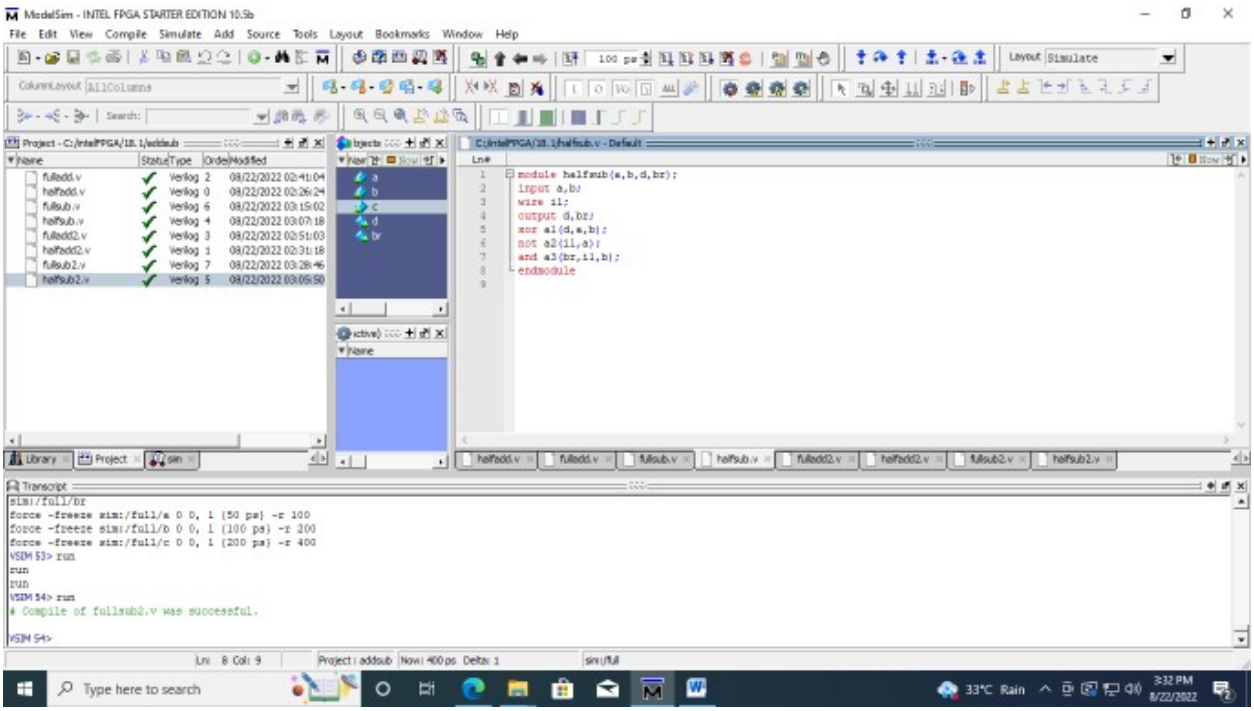
output d,br;

xor a1(d,a,b);

not a2(i1,a);

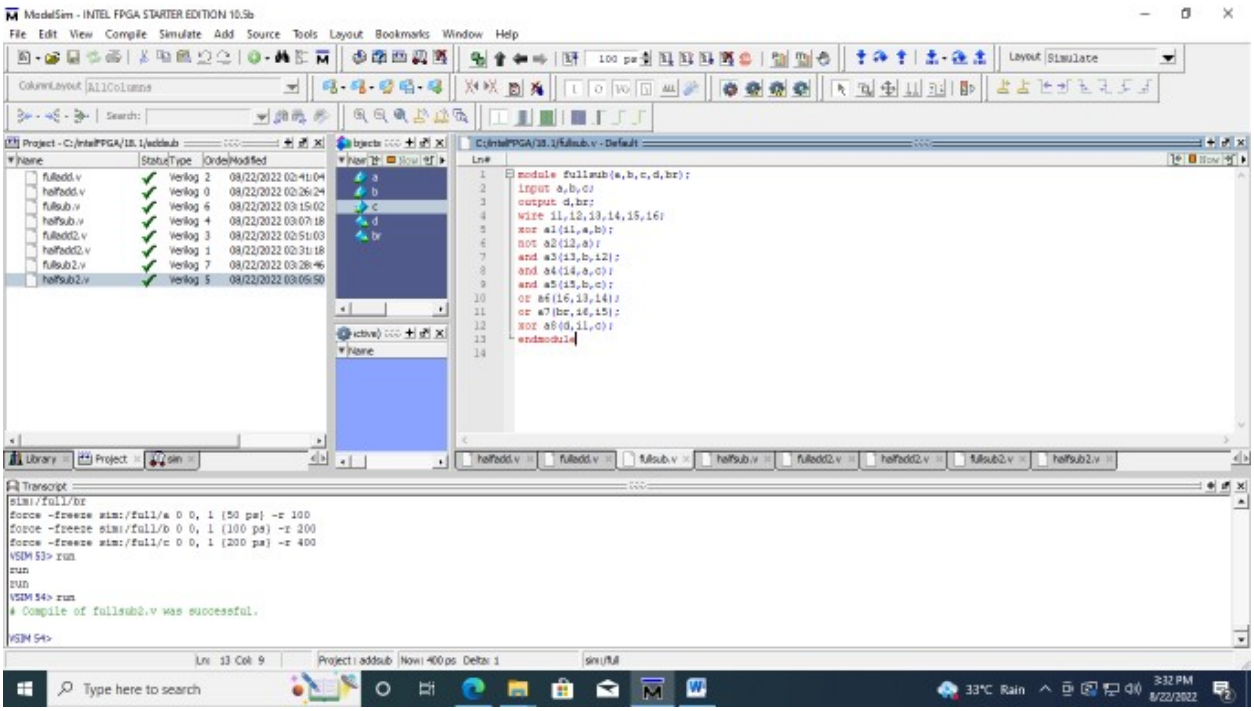
and a3(br,i1,b);

endmodule



Full Subtractor:

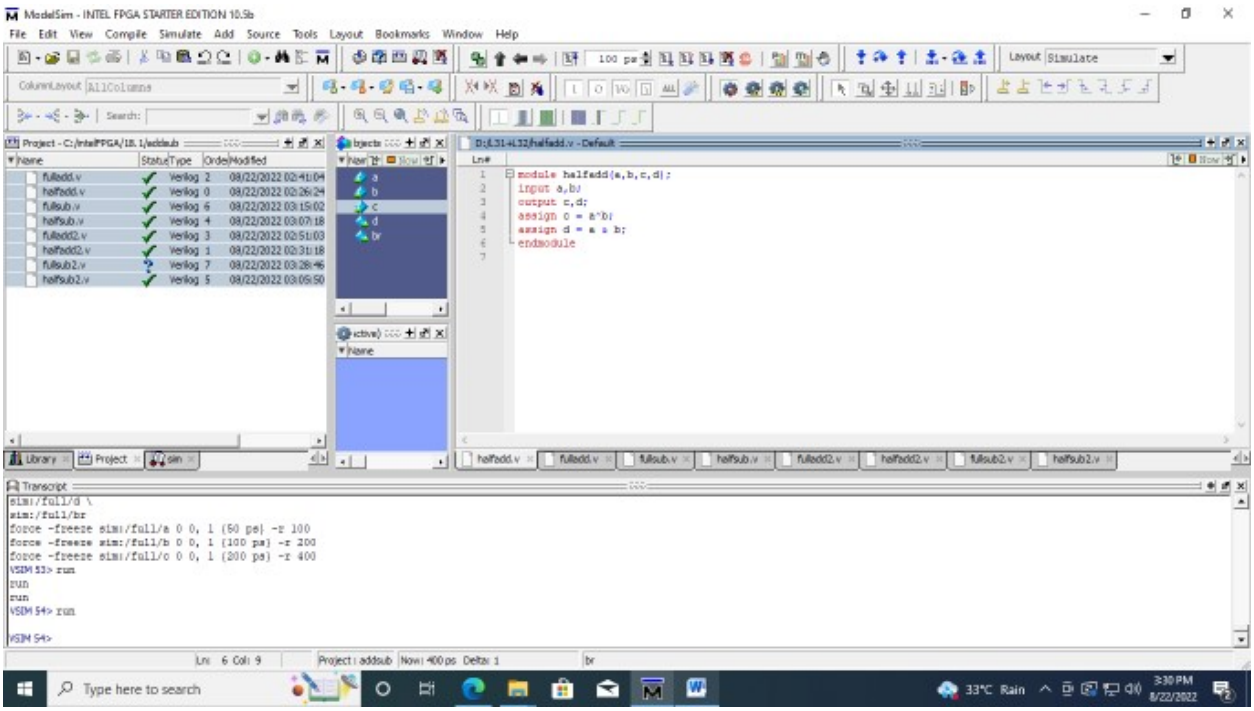
```
module fullsub(a,b,c,d,br);

input a,b,c;

output d,br;

wire i1,i2,i3,i4,i5,i6;

xor a1(i1,a,b);

not a2(i2,a);

and a3(i3,b,i2);

and a4(i4,a,c);

and a5(i5,b,c);

or a6(i6,i3,i4);

or a7(br,i6,i5);

xor a8(d,i1,c);

endmodule
```



<u>Data Flow Modelling:</u>

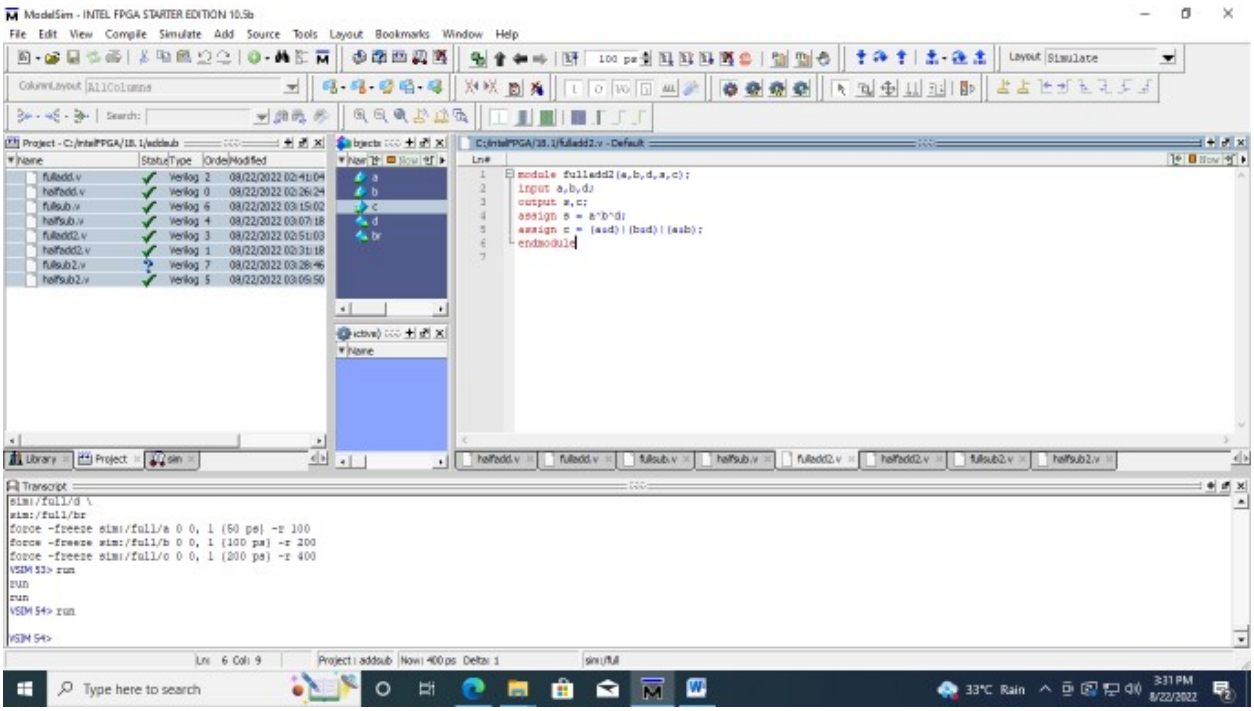
Half Adder:

```
module halfadd(a,b,c,d);

input a,b;

output c,d;

assign c = a^b;

assign d = a & b;

endmodule
```



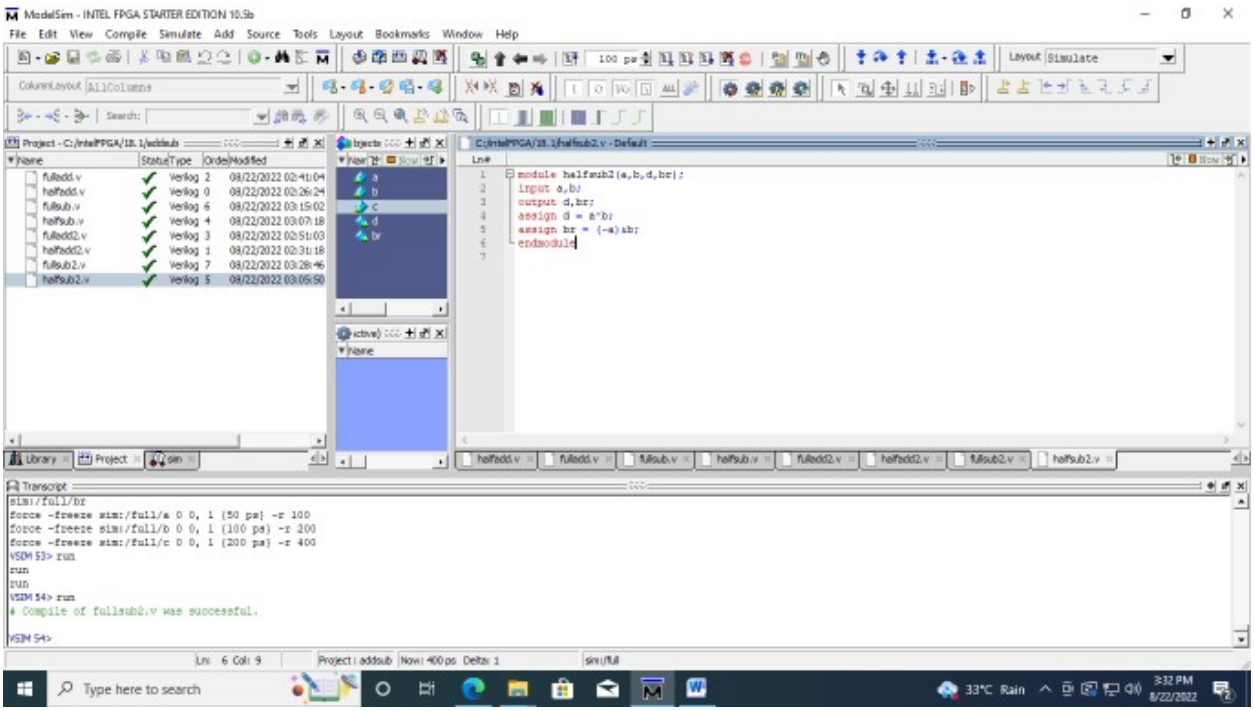
Full Adder:

```
module fulladd2(a,b,d,s,c);

input a,b,d;

output s,c;

assign s = a^b^d;

assign c = (a&d)|(b&d)|(a&b);

endmodule
```



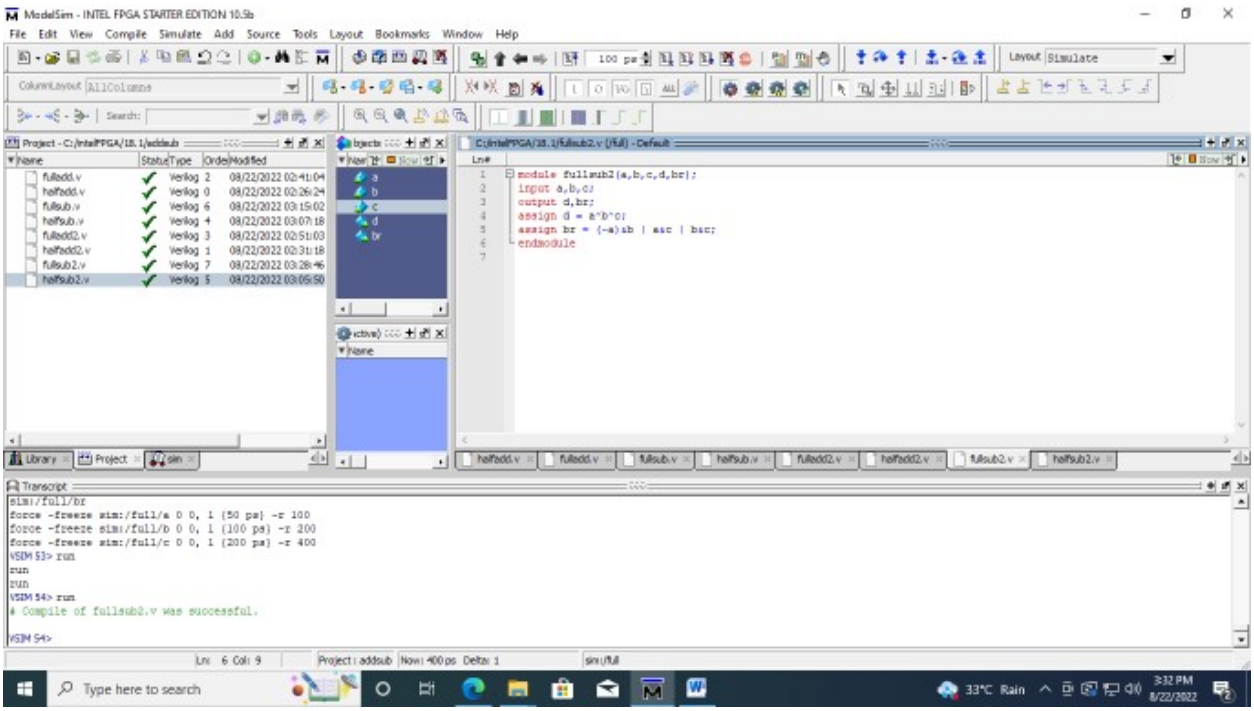
Half Subtractor:

```
module halfsub2(a,b,d,br);

input a,b;

output d,br;

assign d = a^b;

assign br = (~a)&b;

endmodule
```



Full Subtractor:

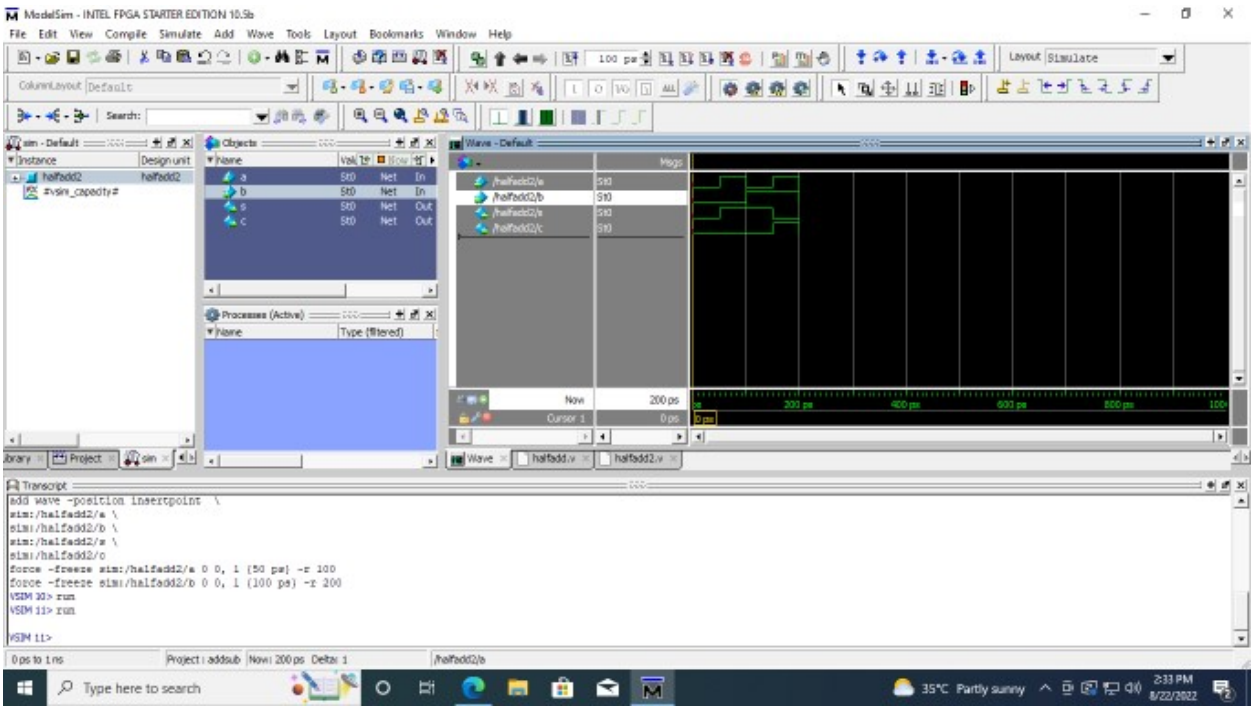
```
module fullsub2(a,b,c,d,br);

input a,b,c;

output d,br;

assign d = a^b^c;

assign br = (~a)&b | a&c | b&c;

endmodule
```



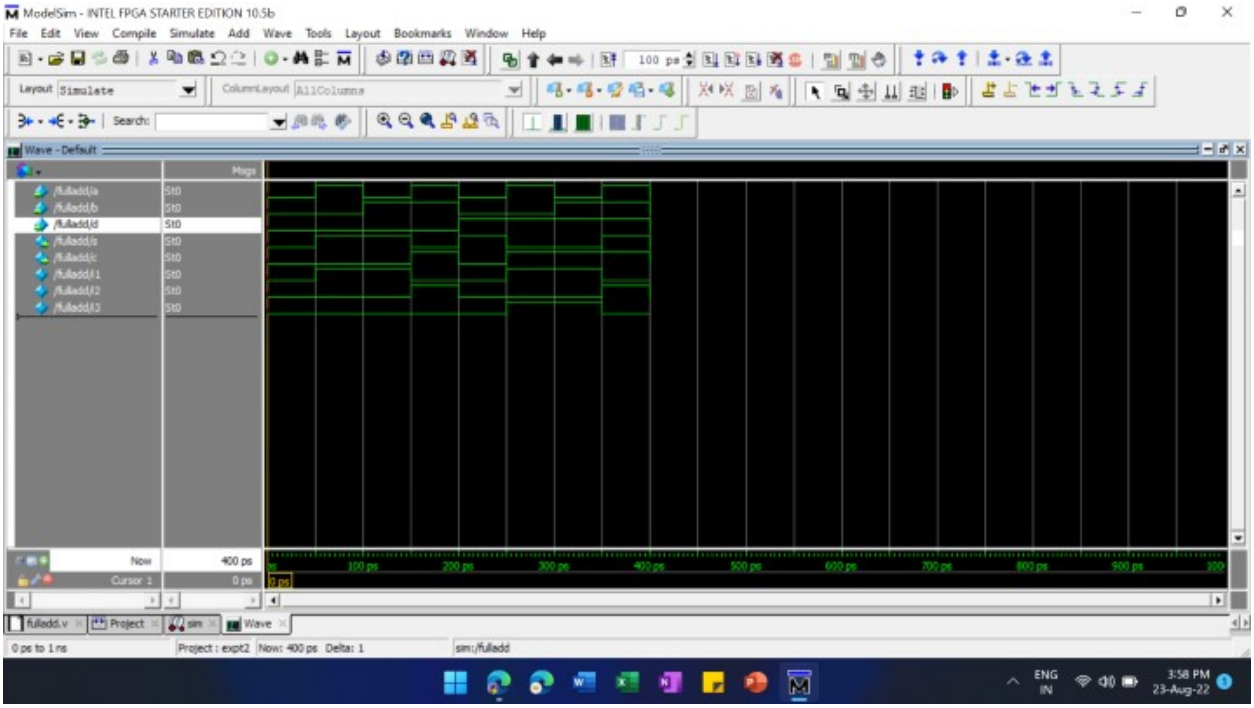
Verilog Output:

<u>Gate Modelling:</u>

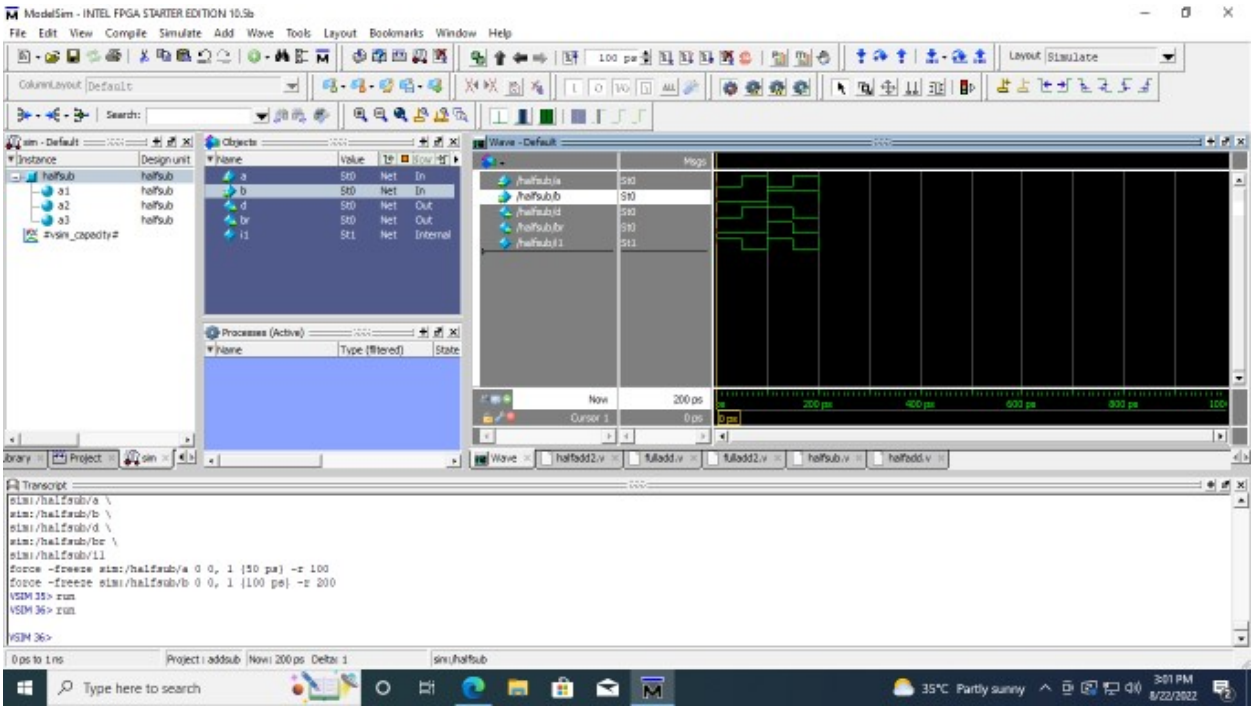
Half Adder:



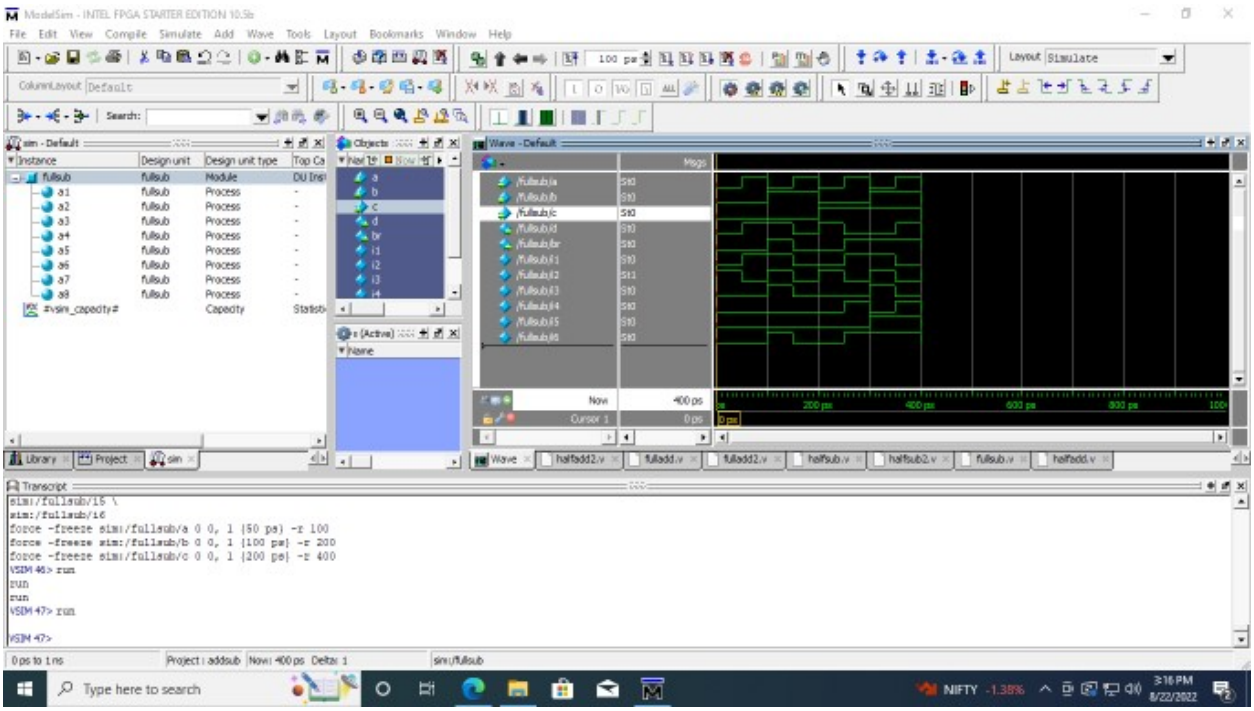
Full Adder:



Half Subtractor:

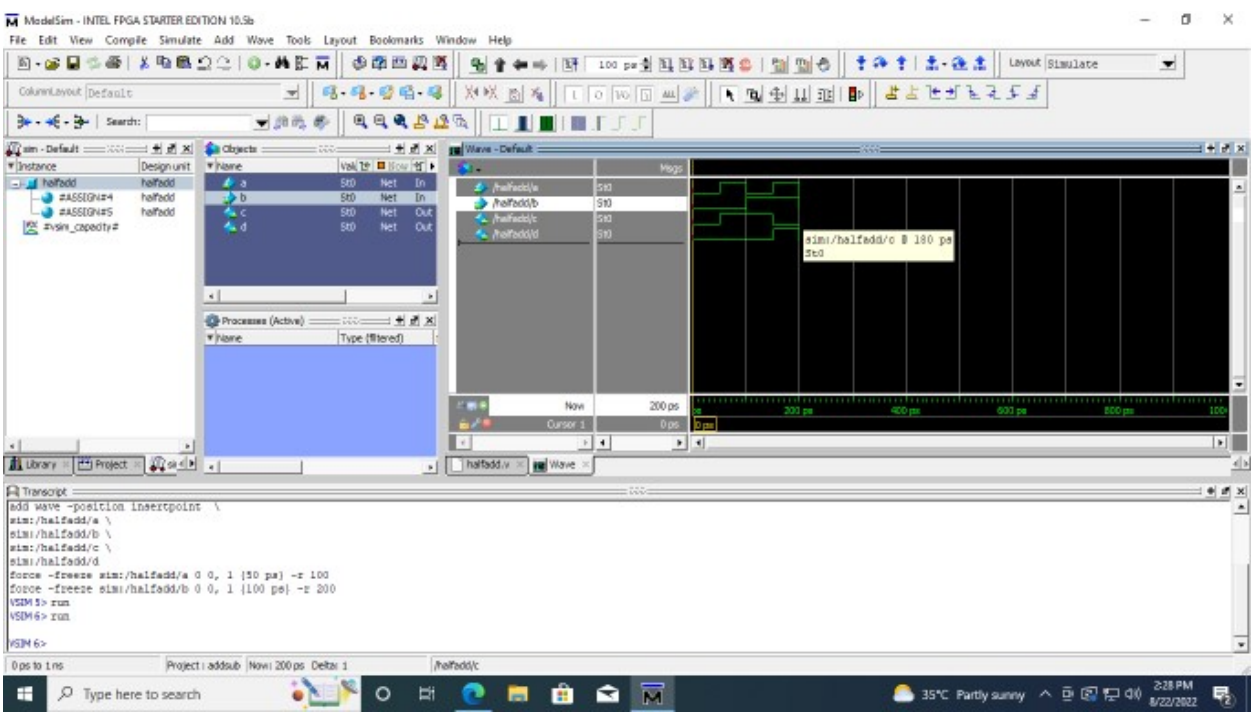


Full Subtractor:

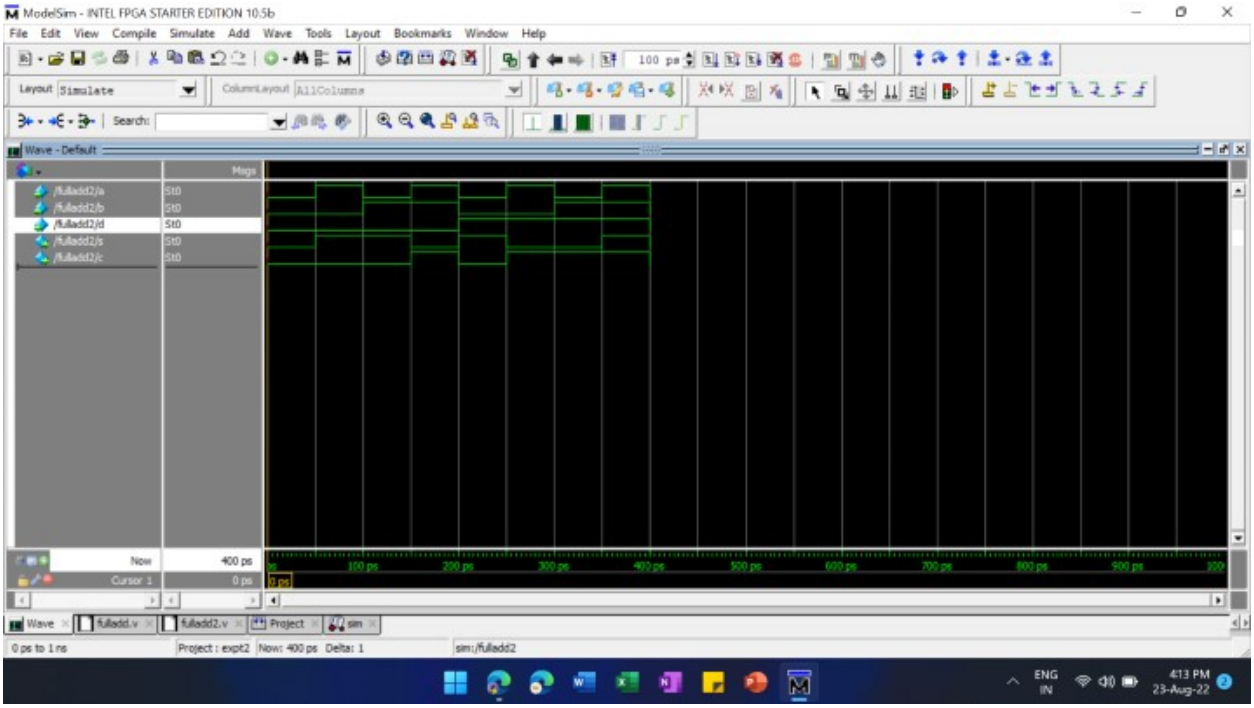


Data Flow Modelling:

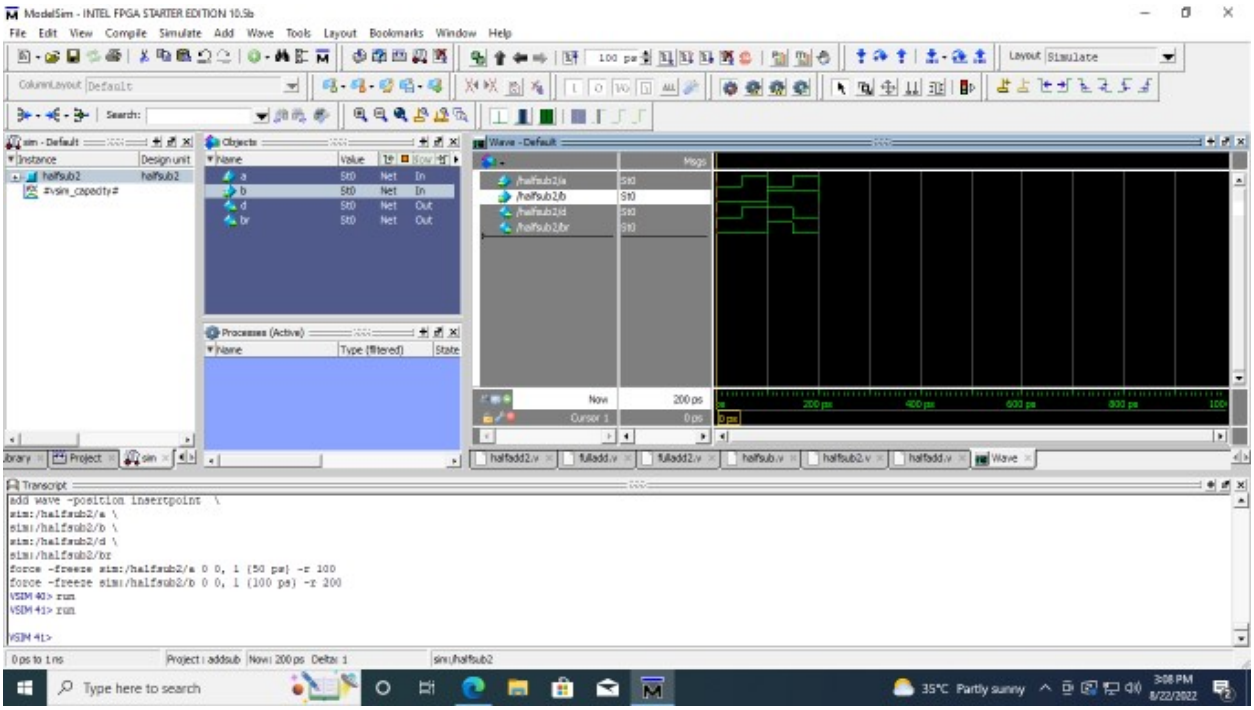
Half Adder:



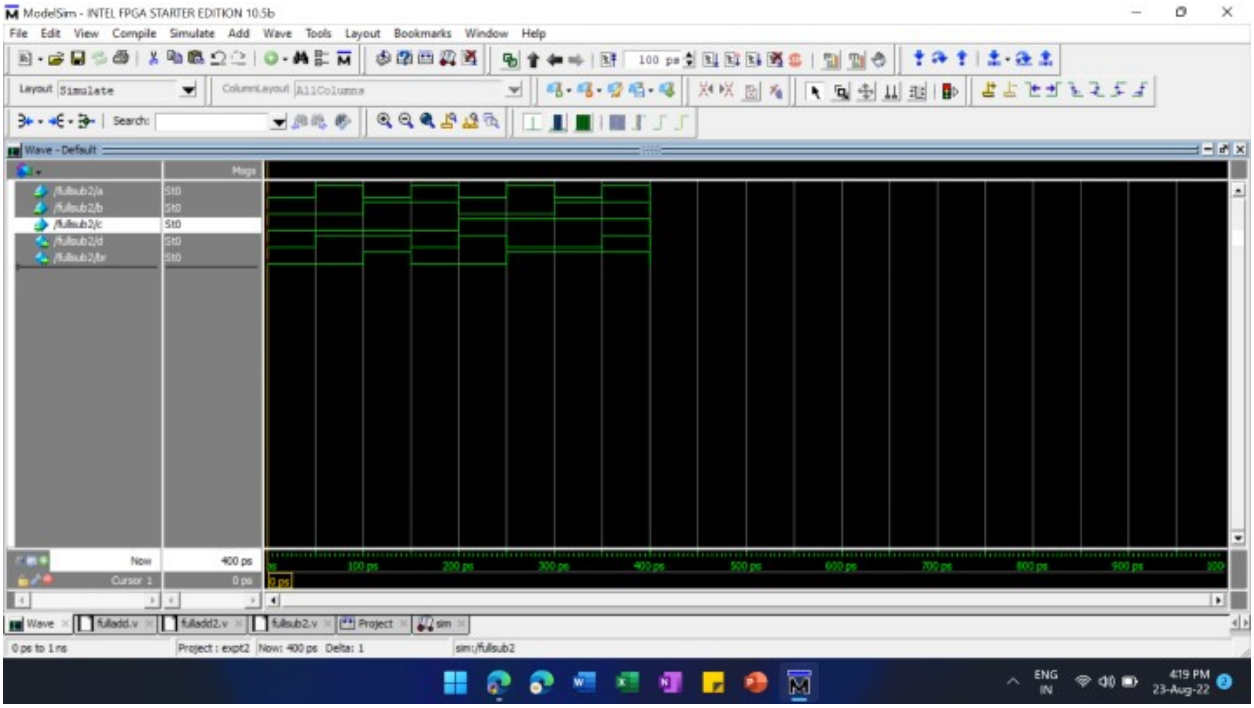
Full Adder:



Half Subtractor:



Full Subtractor:



Result:

The adder and subtractor circuit were designed and constructed using logic gates in verilog and their truth table was verified.